FINAL-YEAR ENGINEERING PROJECT

CareerCompass AI

Live Job Market Analytics, Career Recommendation, Skill-Gap Analysis and Personalized Learning Roadmap System

PROJECT PRESENTATION GUIDE

A beginner-friendly blueprint for explaining, building, testing, and demonstrating a real-data career guidance application.

Project in one sentence

CareerCompass AI turns permitted job-posting data into transparent market insights, compares a student's skills with job-role requirements, and suggests practical next steps for learning.

AT A GLANCE	DESIGN CHOICE
Data source	Authorized job-search API; first candidate: Adzuna, subject to access and account terms.
Backend	Python + FastAPI for API integration, validation, and analysis services.
Data / AI	Pandas, NumPy, editable skill dictionary, explainable content-based matching.
Storage / UI	MySQL for structured data; React + Vite for pages; Plotly charts.
Operating model	Periodic refresh; distinguish source posting date from collection time.

Use this PDF as a presentation script and project design reference. The phases are staged: prove the data pipeline and analytics before adding recommendations and roadmaps.

1. Problem, aim, and objectives

Students often choose career paths using incomplete or scattered information. Job descriptions contain useful signals about skills, locations, roles, and sometimes salary, but those signals are difficult to compare manually. CareerCompass AI organizes these signals and relates them to a student's stated profile.

Aim

Build a working web application that analyzes permitted job-posting data and provides explainable career suggestions, skill-gap priorities, and a personalized learning plan.

Project objectives

- Collect job listings through a documented, authorized API and retain source and refresh metadata.
- Clean and deduplicate records; extract skills using a transparent, editable dictionary.
- Summarize unique postings and calculate skill demand as a percentage of unique jobs.
- Compare self-reported student skills with role requirements and explain recommendations.
- Turn prioritized skill gaps into weekly learning milestones with trustworthy resources and practice tasks.
- Expose validated REST endpoints and provide a responsive, testable interface.

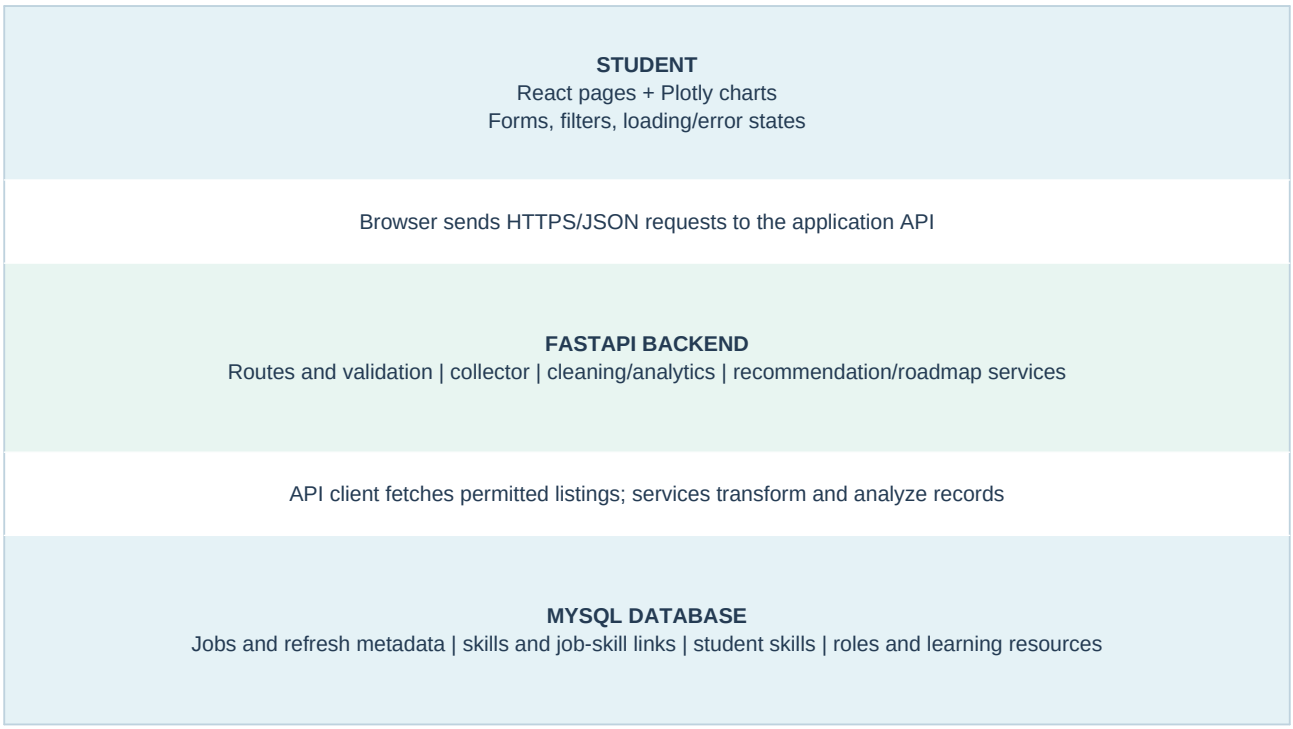
Scope and boundaries

This is a decision-support and learning-planning tool, not a hiring predictor, recruitment decision system, or guaranteed career outcome. Data coverage depends on API access, market, account permissions, and returned fields. A match score is a transparent similarity indicator, never a hiring probability.

> **Important project claim**
>
> Describe the dashboard as periodically refreshed. Call it live only if an implemented refresh process actually retrieves new source data at the time of use.

2. System architecture

Think of the system as layers. The frontend asks the backend for data; the backend handles provider access, business rules, validation, and database operations; the database stores reusable records.



How to explain the flow

1) A student opens the dashboard or submits a profile. 2) React calls a FastAPI endpoint. 3) FastAPI reads prepared records or invokes a service. 4) The service uses stored job data and validated rules. 5) FastAPI returns JSON. 6) React displays results and explains their source and limits.

Why separate layers?

The collector can change without rewriting charts; cleaning logic can be tested without a browser; and the interface does not need database credentials. This separation makes the project easier to maintain.

Presentation phrase

The backend is the trusted middle layer: the browser never calls the job provider with a secret API key, and it never connects directly to MySQL.

3. Job collection and data pipeline

The pipeline is the foundation. Every later chart or recommendation is only as reliable as the data behind it.

STEP	WHAT HAPPENS	RELIABILITY CHECK
Configure	Read provider credentials and country/settings from backend environment variables.	Fail clearly on missing configuration; never commit secrets.
Request	Call documented search endpoint with timeouts, pagination, and bounded retries.	Handle HTTP errors, rate limits, malformed responses, and empty pages.
Preserve	Store allowed source fields and provider ID as raw/source records.	Keep source URL, provider, posted_date, and collected_at distinct.
Normalize	Standardize text, whitespace, titles, locations, and missing values.	Do not invent missing company, date, or salary values.
Deduplicate	Prefer provider ID; fallback to normalized title, company, location, and date.	Track duplicate counts; avoid merging clearly different postings.
Extract	Match job text/title against editable skill dictionary and aliases.	Test aliases and reduce substring false positives.
Report	Produce data-quality summary and refresh outcome.	Record attempted/successful time, counts, and failures.

Example data-quality report

A refresh report should state received, valid, duplicate, and saved counts; missing-field rates; invalid dates; salary coverage; source and date range; and last successful refresh. Report observed values only; do not use fabricated counts in a final presentation.

Data handling rule

Keep raw/source records separate from cleaned records. This preserves traceability between provider output and normalized data.

4. Database design

A relational design links job listings to normalized skills and student profiles to self-reported skills. Use primary/foreign keys, uniqueness constraints, and indexes for filtered fields.

TABLE	PURPOSE / IMPORTANT FIELDS
Jobs	Provider, external_id, title, company, location, description snippet, salary fields/currency, posted_date, source_url, collected_at. Unique provider + external_id when available.
Skills	Canonical skill names, aliases, and category (technical or soft). Canonical name is unique.
JobSkills	Many-to-many link between jobs and skills; composite unique key (job_id, skill_id).
StudentProfiles	Minimal optional label, education/experience summaries, interests, preferred location/roles. Avoid unnecessary sensitive data.
StudentSkills	Many-to-many profile-skill link; include level only if it is needed and defined.
CareerRoles	Role name and description; connect role to requirements and importance weights.
LearningResources	Title, related skill/role, provider, verified URL, type, last_checked_at.
LearningRoadmaps	Profile/role, generated_at, milestones, progress. A RoadmapMilestones table may normalize weekly items.

Relationships to explain

Jobs and Skills have a many-to-many relationship through JobSkills: a job mentions several skills and a skill appears in many jobs. CareerRoles should similarly use a CareerRoleSkills join table so requirements and weights are not stored as comma-separated text.

Useful indexes and privacy

Index provider/external ID, posted date, location, title, and join-table foreign keys. Constrain invalid salary ranges and duplicates. Define retention deliberately and store only student data the features need.

5. Job market analytics dashboard

Every dashboard panel should disclose its source, selected filters, sample size, and covered posting-date range where the data supports it.

INDICATOR	DEFINITION / DISPLAY NOTE
Unique job postings	Count deduplicated postings within selected filters and data window.
Skill demand	100 x unique selected jobs mentioning skill / all unique selected jobs. Show denominator and date window.
Openings by city/title	Counts by normalized location and title; disclose Unknown/Other values.
Experience distribution	Use only when a reliable experience field is available; show unknown share.
Salary summaries	Use comparable reliable values; preserve currency and flag predicted/estimated values.
Trends over time	Requires dated records or repeated snapshots. One current snapshot is not a historical trend.
Refresh status	Show last successful data refresh and provider; do not confuse it with posting date.

Filter behavior and chart ideas

Location, role/title, experience, and skills should update the same selected dataset. Skill percentages recalculate with the filtered unique-job denominator. Useful displays include KPI cards, sorted skill/title bars, a location chart only when resolution is adequate, and salary histograms only for comparable samples. Show an empty state when no postings match.

Example interpretation

In [date range], [skill] appeared in [count] of [sample size] unique listings ([percentage]%). Source: [provider], retrieved [refresh time]. This describes the collected sample, not every job in the labor market.

6. Explainable career recommendation

Begin with content-based matching. Represent each role by documented skill requirements and importance weights. Compare those requirements with skills the student enters. This is auditable and does not require a large labeled hiring dataset.

Transparent weighted coverage

For a role with required skills R and non-negative weights w: match score = 100 x sum of weights for matched skills / sum of weights for all role skills. Show matched skills, missing skills, and weights. An unweighted overlap score is acceptable for the first working version.

Illustrative example (not a measured result)

Suppose a sample role has Python (3), SQL (3), pandas (2), Plotly (1), scikit-learn (2), and statistics (2): total weight 13. A profile with Python, SQL, pandas, and statistics matches weight 10; illustrative coverage is 10/13 = 76.9%. Plotly and scikit-learn are gaps. This illustrates arithmetic only; it is not project evaluation data.

INPUT	HOW IT IS USED
Technical skills	Primary evidence for overlap; normalize aliases to canonical skills.
Education / experience	Optional context or filter; missing values are not negative evidence.
Interests	May reorder or annotate suggestions; disclose the rule.
Preferred location / roles	Use as filters/tie-breakers; disclose excluded matches.

Interpretation

The score measures overlap with encoded role requirements. It is not probability of being hired, an assessment of personal worth, or a guarantee of success.

7. Skill-gap analysis

After a student selects a role, show which requirements they report meeting and what they could learn next. Make the classification rule visible; let students correct their self-reported skills.

CATEGORY	EXAMPLE RULE	USER-FACING MEANING
Possessed	Student reports acquired skill or level meets role threshold.	You report this required skill.
Partial	Related skill, beginner familiarity, or level below role threshold.	You have a foundation; practice toward target level.
Missing	No matching self-reported skill is recorded.	Not listed in your profile; confirm before calling it a gap.

Prioritize, do not overwhelm

A practical priority may combine role importance with observed filtered-market demand. Explain the inputs and tie-break rule. Demand alone does not guarantee a better career outcome.

Useful displays

- Counts or stacked bars for possessed, partial, and missing skills.
- Ranked gap list with role importance, observed job-demand percentage, and short explanation.
- A summary based on actual selected data, for example: "You report X of Y required skills; Z are listed as gaps."
- An edit-profile action so a false missing label can be corrected.

Fairness and uncertainty

Profiles are self-reported and job descriptions are incomplete. Say "not listed in your profile," not "you do not have this skill." Avoid protected attributes in scoring.

8. Personalized learning roadmap

A roadmap translates prioritized gaps into manageable weekly milestones. Begin with curated templates; later, adapt to a student's chosen study time.

WEEK	MILESTONE EXAMPLE	EVIDENCE OF PRACTICE
1	Review a missing skill using an official or curated resource.	Notes and a small exercise.
2	Practice that skill on a small task.	Runnable notebook or code sample.
3	Combine it with a related skill.	Mini-project, README, sample output.
4	Review, reflect, update profile.	Checklist, self-assessment, next step.

Each milestone includes

- Skill and why it matters for the selected role and observed sample.
- Weekly goal, effort estimate if known, and student-updatable progress checkbox.
- Practice task, small project, and clear deliverable.
- Resource link only after URL checking; store provider and last_checked_at. Never fabricate a link.

Verified starter resources

Official reference pages checked while preparing this guide: Python tutorial; pandas getting-started tutorials; scikit-learn documentation; Plotly for Python. Recheck availability and suitability before assigning resources.

Example mini-project

For a data analyst path: clean a small permitted dataset, calculate a defined summary, create one chart, and explain sample limits. This is practice, not evidence of market performance.

9. Application API and user interface

FastAPI provides a documented REST interface. Validated schemas keep the frontend/backend contract explicit.

ENDPOINT EXAMPLE	PURPOSE
GET /api/health	Report whether the backend process is responding.
GET /api/market/summary	Counts, source/window metadata, and last successful refresh.
GET /api/market/skills	Skill-demand percentages with unique-job denominator.
GET /api/market/jobs	Filtered jobs or aggregates with pagination.
POST /api/recommendations	Validate student profile; return ranked roles and score explanation.
POST /api/skill-gaps	Compare profile with role and prioritize gaps.
POST /api/roadmaps	Create roadmap from selected role and gaps.
GET /api/sources	Describe source provider and refresh limitations.

Frontend pages

Home; Job Market Dashboard; Career Recommendation; Skill-Gap Analysis; Learning Roadmap; Data Sources and Project Information. Reuse navigation, filters, chart, status, and error components.

User experience and security

- Show loading feedback and useful errors with retry guidance. Describe empty results and stale data honestly.
- Use accessible labels, keyboard-friendly controls, responsive layouts, and readable chart legends.
- Never send provider credentials to React. Protect backend secrets and avoid leaking credentials through logs.

10. Development phases and test strategy

Build in small increments. Verify each phase before continuing so errors stay local and each milestone can be demonstrated.

PHASE	DELIVERABLE / CHECKPOINT
1. Setup	Python environment, FastAPI health route, configuration, first test.
2. Collection	Authorized API client, pagination, timeouts, error handling, raw records.
3. Cleaning	Normalization, deduplication, skill dictionary, quality report, unit tests.
4. Database	MySQL schema, indexed relations, persistence tests.
5. Analytics	Unique-job and skill-demand APIs, filters, refresh metadata, tests.
6. Dashboard	React/Vite pages, API integration, charts, loading/error UI.
7. Recommendation	Profile inputs, role skills, explainable scores and gap tests.
8. Roadmap	Milestones, curated links, progress controls, verification metadata.
9. Integration	End-to-end checks, README, demo configuration, presentation rehearsal.

Tests that prove behavior

Test deduplication, missing fields, skill aliases/false positives, API pagination/errors, demand denominator, currency handling, filter consistency, score arithmetic, gap categories, invalid inputs, and resource metadata. Mock provider responses in routine tests to avoid API/quota dependency.

Do not invent evaluation results

Report only measured outputs: actual collection window, sample size, coverage, test results, and limitations. If a metric was not measured, label it planned or omit it.

11. Demonstration and viva preparation

Suggested 5-minute demonstration

TIME	DEMO ACTION	WHAT TO EXPLAIN
0:00-0:40	State problem and objective.	Connect market signals to student learning decisions.
0:40-1:30	Show source, refresh, and filters.	Authorized API, periodic refresh, date range, and sample limit.
1:30-2:15	Show skills and role charts.	Demand denominator is unique filtered postings, not raw mentions.
2:15-3:20	Enter a profile and request recommendations.	Score formula, matched/missing skills; not hiring probability.
3:20-4:20	Open gap analysis and roadmap.	How priorities lead to goals, practice, and checked resources.
4:20-5:00	State limitations and next steps.	Coverage, partial descriptions, self-reporting, periodic freshness.

Common viva questions

QUESTION	SHORT ANSWER
Why FastAPI?	Python API framework with validation, generated docs, and clear separation from React.
Why content-based recommendation?	Interpretable and useful before labeled training data exists.
How is skill demand calculated?	Unique selected postings mentioning a skill / all unique selected postings x 100.
How avoid misleading trends?	Require dates or repeated snapshots and disclose the observation window.
Main limitations?	Provider coverage/access, partial descriptions, noisy extraction, self-reporting, changing demand.

Official references

Adzuna API overview | Adzuna search docs | FastAPI tutorial | React Learn | Python tutorial | pandas | scikit-learn | Plotly. Recheck provider country availability, quotas, account terms, and permitted use before implementation.

Final takeaway

CareerCompass AI makes the path from source data to student action visible: where the data came from, how it was measured, why a role was suggested, and what the student can learn next.

Build the pipeline first. Measure honestly. Explain every score. Keep the student in control.